

Contents

Rapport final - Projet Heapsort	1
1. Introduction	1
2. Présentation du sujet	1
3. Répartition des tâches	1
4. Implémentation réalisée	2
5. Organisation du projet	2
6. Tests et validation	3
7. Résultats expérimentaux	3
8. Avantages du Heapsort	3
9. Conclusion	4
10. Commandes utiles	4
11. Remerciements	4

Rapport final - Projet Heapsort

1. Introduction

Ce projet a pour objectif de mettre en œuvre l'algorithme de tri par tas, appelé Heapsort, ainsi que les structures associées nécessaires à son fonctionnement : le tas binaire, la construction du tas en temps linéaire et une file de priorité simple. Le travail a été réalisé en Python, sans l'utilisation de bibliothèques externes de structures de données.

Le choix de ce sujet est motivé par l'intérêt pédagogique de l'algorithme Heapsort : il offre une complexité garantie en $O(n \log n)$ dans tous les cas, contrairement à certains algorithmes comme Quicksort, qui peuvent dégénérer en $O(n^2)$ sur certaines entrées.

2. Présentation du sujet

Le tri par tas repose sur une structure appelée tas binaire. Cette structure est représentée sous forme de tableau Python, ce qui permet de conserver un arbre binaire complet tout en utilisant un espace mémoire très faible.

Les notions essentielles abordées dans ce projet sont : - la représentation d'un tas sous forme de tableau ; - les opérations d'entassement vers le bas et vers le haut ; - la construction d'un tas en temps linéaire ; - l'algorithme Heapsort ; - l'utilisation du tas pour une file de priorité.

3. Répartition des tâches

Tâche	Description	Responsable
T1	Structure de tas et entassement	Rajo
T2	Insertion et extraction	Mampita
T3	Construction du tas en $O(n)$	Alex
T4	Implémentation complète de Heapsort	Feno

Tâche	Description	Responsable
T5	File de priorité et visualisation	Meddy
T6	Tests, benchmarks, rapport et vidéo	Mihajaso

4. Implémentation réalisée

4.1 Structure de tas binaire

Le tas binaire est représenté par une liste Python. Les relations entre nœuds sont définies par les formules suivantes : - $\text{parent}(i) = (i - 1) // 2$ - $\text{enfant_gauche}(i) = 2i + 1$ - $\text{enfant_droit}(i) = 2i + 2$

Le projet implémente les opérations de base suivantes : - entasser vers le bas ; - entasser vers le haut ; - insertion d'une valeur ; - extraction du maximum.

4.2 Construction du tas en temps linéaire

Au lieu de construire le tas par insertions successives, ce qui coûterait $O(n \log n)$, l'algorithme utilisé parcourt tous les nœuds internes à partir du dernier parent jusqu'à la racine. Cette méthode permet de construire le tas en $O(n)$.

4.3 Heapsort

L'algorithme Heapsort fonctionne en deux étapes : 1. construction d'un max-tas à partir du tableau ; 2. extraction répétée du maximum pour placer les valeurs dans l'ordre croissant à la fin du tableau.

Cette approche garantit une complexité en $O(n \log n)$ dans tous les cas, ce qui en fait un algorithme robuste et fiable.

4.4 File de priorité

Le tas sert également de base à une file de priorité simple. L'élément de priorité maximale peut être extrait rapidement, ce qui confirme l'utilité pratique de cette structure.

5. Organisation du projet

Le projet est organisé comme suit :

```
projet-heapsort/
├── src/
│   ├── heap.py
│   ├── heap_ops.py
│   ├── build_heap.py
│   ├── heapsort.py
│   └── priority_queue.py
├── tests/
└── benchmarks/
```

├─ rapport/
└─ video/

Les fichiers principaux sont : - [src/heap.py](#) : implémentation du tas et des opérations de base ; - [src/build_heap.py](#) : construction du tas en $O(n)$; - [src/heapsort.py](#) : algorithme Heapsort ; - [benchmarks/comparaison_tris.py](#) : comparaison avec Quicksort et Tri fusion.

6. Tests et validation

Le projet a été testé de manière approfondie. Les tests couvrent : - les cas élémentaires (tableau vide, un seul élément) ; - les tableaux déjà triés ou triés à l'envers ; - les tableaux contenant des doublons ; - les tableaux avec des valeurs négatives ; - des grands tableaux aléatoires.

La validation exécutée a donné les résultats suivants : - 43 tests passés avec succès via pytest ; - exécution correcte de l'algorithme Heapsort sur plusieurs exemples ; - génération réussie du graphique de comparaison des performances.

7. Résultats expérimentaux

Le benchmark réalisé compare Heapsort, Quicksort et Tri fusion sur des tableaux de tailles croissantes.

Taille du tableau	Heapsort	Quicksort	Tri fusion
100	0.000199 s	0.000129 s	0.000166 s
500	0.001055 s	0.000710 s	0.001005 s
1000	0.002606 s	0.001240 s	0.001879 s
2000	0.005766 s	0.002939 s	0.004122 s
5000	0.016368 s	0.007909 s	0.010576 s
10000	0.039210 s	0.016963 s	0.025748 s

Le graphique généré est disponible dans [benchmarks/comparaison_tris.png](#).

Analyse des résultats

On observe que Quicksort est souvent légèrement plus rapide sur les petites et moyennes tailles, mais Heapsort reste très compétitif et surtout plus robuste car il possède un pire cas garanti en $O(n \log n)$. Le Tri fusion est également efficace, mais consomme davantage de mémoire en raison de la création de sous-tableaux temporaires.

8. Avantages du Heapsort

Le principal avantage de Heapsort est sa garantie de performance. Contrairement à Quicksort, il ne tombe pas dans un pire cas catastrophique sur des données mal choisies. De plus : - il trie en place ; - il n'utilise pas de mémoire supplémentaire importante ; - il est relativement simple à implémenter à partir d'un tas binaire.

9. Conclusion

Ce projet a permis de mieux comprendre le fonctionnement d'un tas binaire et de l'algorithme Heapsort. Il a aussi montré l'intérêt de choisir un algorithme selon ses garanties de performance et de complexité. Le travail réalisé est complet, testé et prêt pour la démonstration finale.

10. Commandes utiles

Pour exécuter les tests :

```
python -m pytest -q tests
```

Pour lancer le benchmark :

```
cd benchmarks  
python comparaison_tris.py
```

Pour tester rapidement Heapsort :

```
python -c "from src.heapsort import heapsort; t=[4,10,3,5,1,8,6]; heapsort(t); print(t)"
```

11. Remerciements

Merci à l'équipe ayant contribué à la réalisation de ce projet.